

Optimization of an Embedded Visual Perception Pipeline (SuperPoint + LightGlue)

1) Introduction

Deploying state-of-the-art computer vision algorithms on resource-constrained embedded systems (Edge AI) is one of the major technological bottlenecks of autonomous robotics, intelligent vehicles, and mobile devices. In the context of navigation, Simultaneous Localization and Mapping (SLAM), or visual odometry, extracting and matching stable geometric keypoints from one image to the next forms the fundamental basis of spatial perception.

Traditionally, this processing runs in two asymmetric blocks:

- **The extractor (Front-End):** responsible for detecting distinctive geometric points and assigning each one a unique numerical signature called a descriptor.
- **The matcher (Back-End):** responsible for solving the geometric matching problem and removing false correspondences (outliers).

The central objective of this project is to optimize the state-of-the-art reference pipeline, composed of SuperPoint for extraction and LightGlue for matching.

This report documents the approach I followed over the course of the semester to optimize this pipeline using several methods such as quantization, pruning, and knowledge distillation. We will see that I encountered several difficulties and which methods I used to resolve them.

2) Explanation of SuperPoint

SuperPoint is a CNN designed to process images in a single pass — that is, it simultaneously extracts two pieces of information: the location of interest points and their descriptors, whereas these two tasks were previously handled separately. To achieve this, it uses a shared encoder to avoid computing the image features twice. Thanks to this mechanism, the model can reach a speed of around 70 frames per second. The encoder used is VGG-like (it is not a pretrained VGG). It is composed of 8 convolutional layers organized in pairs of two, with a channel structure of 64-64-64-64-128-128-128-128. Every two layers, a 2×2 max-pooling operation is applied. In total, the image is therefore downsampled by a factor of 8. Thus, after passing through the encoder, each pixel of this compressed map corresponds to an 8×8 pixel cell of the original image.

The point where SuperPoint truly stands out is the use of two separate decoders: one dedicated to interest-point detection and the other to description. For the interest-point decoder, the network does not directly produce a map the size of the original image. It generates a tensor of size $H_c \times W_c \times 65$ (where H_c and W_c correspond to the image dimensions divided by 8 after the encoder). The first 64 channels correspond to the possible point positions within each 8×8 cell, while the 65th channel, called the dustbin, indicates that no interest point is present in that cell. To recover the original resolution, a softmax is first applied over the 65 channels, then the dustbin channel is removed. Next, a sub-pixel convolution (or pixel shuffle) operation is used to rearrange the values and recover a map of size $H \times W$.

The descriptor decoder, meanwhile, computes a single descriptor per 8×8 cell, which corresponds to a semi-dense grid. This choice reduces both the memory used and the computation time. The network then applies bicubic interpolation to obtain a dense $H \times W$ grid, this method taking into account the 16 nearest neighbors (unlike bilinear interpolation, which considers only 4). Finally, to optimize comparisons, L2 normalization is applied: each vector is

divided by its Euclidean norm. This way, the comparison between two descriptors can simply be done via a dot product, a very fast operation on a processor.

Regarding training, it takes place in three crucial stages. First, since there is no real-image dataset containing millions of perfectly annotated interest points, the authors start by creating a controlled artificial environment. They generate a dataset of simple geometric shapes such as triangles, quadrilaterals, cubes, or stars. In this context, the exact position of each interest point is known, which provides a reliable ground truth. Homographic deformations are then applied to augment the data. At this stage, only the detector part of the network is used (the encoder and the detection head). This sub-model is called MagicPoint, and it learns to detect these synthetic interest points.

However, after this training, MagicPoint performs very well on simple shapes but much less well on real images. To address this problem, the authors introduce Homographic Adaptation. The principle relies on geometric consistency: a good detector must be covariant, meaning that when an image is transformed, the detected points must undergo the same transformation. Concretely, $N_h = 100$ random homographies (rotations, zooms, perspective changes) are applied to a real image. Each transformed image is passed through MagicPoint, and the results are then reprojected back into the original image's space and overlaid. If a point appears consistently across the majority of the transformations, it is validated as a “real” interest point. The model thus generates its own annotations, enabling self-supervised training.

Through this process, a set of real images accompanied by pseudo-annotated interest points, called pseudo-ground-truth labels, is obtained. It then becomes possible to train the complete model, including both detection and description. To do this, an image I is taken and a random homography H is applied to obtain an image I' . The geometric correspondence between the pixels of I and I' is therefore perfectly known. The network must then minimize a total loss L , which is the sum of two losses. The detection loss L_p relies on a cross-entropy computed over the 65 channels of each 8×8 cell. The descriptor loss L_d is a hinge loss with a positive margin of 1 and a negative margin of 0.2.

In terms of hyperparameters, a weight $\lambda = 0.0001$ is used to balance the detection and description losses. A weighting term $\lambda_d = 250$ is also introduced in the descriptor loss to compensate for the fact that there are far more negative pairs than positive ones. Optimization is performed using the ADAM optimizer with a learning rate of 0.001 and batches of 32 images. Finally, jointly training detection and description allows the two tasks to help each other: knowing what makes a point distinctive improves its localization, and conversely, better localization facilitates learning more discriminative descriptors.

3) First Optimization Approach: INT8 Quantization

To begin, I decided to tackle SuperPoint first, as it seemed simpler being a CNN and because it constitutes the entry point of the pipeline. For this, I tested quantization. An initial INT8 quantization was performed, the goal being to divide the model size by 4 (going from Float32 to INT8). In addition, processors are generally better suited to INT8 or FP16 models. Since the ultimate goal is to embed the optimized model, INT8 quantization seemed like a good starting point.

Unfortunately, I ran into several problems with this method. Indeed, there was a blocking issue when exporting to ONNX or TorchScript, with errors such as `TorchExportError` or `GuardOnDataDependentSymNode`. These errors were due to the fact that the number of detected points varies from one image to another, which creates a conflict with tensors, which are static by nature. In addition, the `torch.jit.trace` method froze the internal logic of the INT8-quantized model, making it unable to generalize to other image dimensions.

To solve this problem, I used a manual wrapper. The goal was to ensure that only the pure convolutions of the VGG backbone and the decoders are executed. Thanks to this, a 100% static ONNX graph could be exported. To limit the loss of precision, a static quantization method with calibration was implemented. A calibration `DataLoader` composed of real images was instantiated to make the observers converge and properly freeze the dynamic ranges (scales and zero-points) of the activations.

4) Evaluation Protocol and Key Metrics

To evaluate and compare the models in this project, I used two metrics: repeatability and MLE.

Repeatability corresponds to the proportion of teacher points that have at least one student point located at a distance less than or equal to a fixed uncertainty radius $\epsilon = 3$ pixels. It is calculated using the following formula:

$$\text{Repeatability} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\min_j D_{i,j} \leq \epsilon) \times 100$$

The MLE (Mean Localization Error) measures the average localization error, expressed in sub-pixels, calculated exclusively over the set of geometric correspondences validated below the threshold ϵ . It is calculated using the following formula:

$$\text{MLE} = \frac{\sum_{i=1}^N \min_j D_{i,j} \cdot \mathbb{I}(\min_j D_{i,j} \leq \epsilon)}{\sum_{i=1}^N \mathbb{I}(\min_j D_{i,j} \leq \epsilon)}$$

5) Analysis of INT8 Quantization Results

Now that we have the evaluation metrics, we can analyze the results of the INT8-quantized model:

- **Model size:** In terms of memory footprint, we went from 5 MB for the base SuperPoint to 1.26 MB for the quantized model. This confirms the expected 4x reduction in model size.
- **Performance (MLE):** The MLE result is very satisfactory. We obtain a score of 0.67, meaning that, on average, the localization error is only 0.67 pixel relative to the points detected by the base SuperPoint. The error is therefore particularly low.
- **Performance (Repeatability):** The outlook is more mixed for repeatability. While some loss of precision due to quantization was naturally expected, the score collapses to 18.50%. This means that only 18.50% of the detected points are identical between the base model and the quantized model.

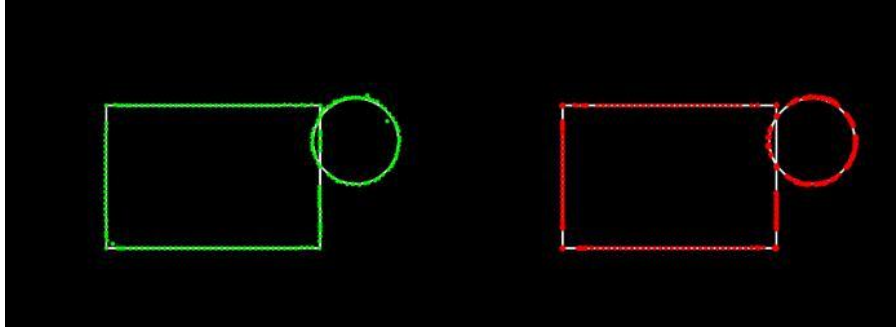


Figure 1 — Points detected by the base model (left, green) vs. the INT8-quantized model (right, red) on simple geometric shapes.

Although the points are very close on average (as shown by the MLE), I decided to change approach in order to try to obtain better repeatability and to find an ideal compromise between reducing model size and the metric scores.

6) Pruning and Knowledge Distillation

The strategy therefore pivoted toward an approach combining pruning and knowledge distillation. For pruning, I opted for structured pruning. This means that I did not remove any convolutional layer from the architecture, but instead chose to reduce the number of filters in the CNN encoder's layers. The lightened model is then trained to imitate a teacher model, which in this case is the base SuperPoint.

The hypothesis is that some channels of the CNN are redundant or less important than others; removing them should therefore have an acceptable impact on performance given the gains obtained in computation time and model size. For this approach, we chose not to apply quantization, meaning the model keeps its weights in Float32 (FP32) format.

Regarding the choice of student model (the pruned model that we are going to train), we test two configurations: a first model with 50% of channels pruned, and a second with 25% of channels pruned.

For the 25% configuration, for example, the number of filters in each encoder layer is multiplied by a coefficient of 0.75. Thus, a convolutional layer that initially had 128 filters now retains only 96. This reduction logic is applied uniformly across all encoder blocks.

Although the encoder's capacity is reduced by pruning, the description heads (convDa and convDb) are designed to project the features to a final dimension strictly maintained at $D = 256$ channels. This architectural specificity guarantees that the student's descriptor space retains the same structural dimensionality as the teacher's. This helps avoid any format incompatibility during the future integration of the matcher (LightGlue).

Unfortunately, as soon as this method was implemented, a problem arose: the student model was “mute.” In other words, it detected no points in the image, and the heatmaps consisted of 98% empty pixels. This was due to the fact that the loss used, MSE (Mean Squared Error), led the student model to converge to a trivial solution. Thus, it detected nothing, which, mathematically, still allowed it to minimize its loss.

7) Designing the Hybrid Loss Function

To solve this new problem, I implemented a Hybrid Loss. The first component of this function is as follows:

1. Detection Loss ($\mathcal{L}_{\text{det}}$)

Instead of a raw MSE, a Softmax function is applied over the 65 channels of the teacher's tensor to obtain its probability distribution P_t . The cross-entropy is then computed with the log-softmax distribution of the student P_s .

The formula for this detection loss is expressed as follows:

$$\mathcal{L}_{\text{det}} = \frac{1}{B \cdot H_c \cdot W_c} \sum_{b=1}^B \sum_{i=1}^{H_c} \sum_{j=1}^{W_c} \sum_{c=1}^{65} -P_T(b, c, i, j) \cdot \log(P_S(b, c, i, j))$$

2. Description Loss ($\mathcal{L}_{\text{desc}}$)

To constrain the directional alignment of the student's descriptors with the teacher's latent space without introducing geometric drift, a loss based on average cosine similarity is used.

It is calculated as follows:

$$\mathcal{L}_{\text{desc}} = 1 - \frac{1}{B \cdot H_c \cdot W_c} \sum_{b=1}^B \sum_{i=1}^{H_c} \sum_{j=1}^{W_c} \text{CosineSimilarity}(D_S(b, :, i, j), D_T(b, :, i, j))$$

3. Total Loss ($\mathcal{L}_{\text{total}}$)

The final loss function combines the two preceding components. An aggressive weighting coefficient $h = 20$ is applied to emphasize the importance of descriptor learning.

The formula for the total loss is expressed as follows:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + 20 \cdot \mathcal{L}_{\text{desc}}$$

8) Performance Analysis of the Pruned Models

Results for the 50%-pruned model

For the 50%-pruned model, training was carried out over 15 epochs, allowing the loss to drop from 6.5790 to 1.1968. Beyond 15 epochs, the loss started to rise again, indicating the onset of overfitting.

However, a problem remained: the student model showed lower confidence in its predictions than the base (teacher) model. To fix this, I searched for the optimal detection threshold. To find the ideal threshold, I used an algorithm that automatically tests about a hundred different values on the test image. It counts how many points the student model detects at each trial and precisely selects the threshold that lets it isolate the exact desired number of points; it was set to 0.01169.

After applying this threshold, I evaluated the model and obtained the following results. For the number of detected points, the Teacher model finds 200 versus 195 for the Student model. Repeatability reaches a score of 45.50% and precision (MLE) stands at 0.3648 pixel, with a size of 2.16 MB (a 56.80% reduction in size).

These results show a clear improvement over INT8 quantization. The number of detected points is nearly identical to that of the base model, the localization error (MLE) remains extremely low (below half a pixel), and repeatability rises significantly.

Results for the 25%-pruned model

These results show us that the pruning technique combined with distillation has higher potential than quantization. Indeed, looking at the MLE score, we see that even when the student model does not detect exactly the same point as the teacher model, it is in fact only 0.3648 pixel away from it on average, while showing much higher repeatability than the INT8-quantized model.

To determine whether it was possible to obtain better results, and although the performance of the previous model was already very satisfactory, I wanted to test the same technique with 25% pruning, still in FP32. For this one, I trained over 18 epochs and ran two separate tests.

First, on simple geometric shapes such as squares or triangles (as with the INT8 model, see Figure 1), the 25%-pruned model performed very well. It reached a high repeatability rate of 82.66% and a mean localization error (MLE) of only 0.6625 pixel, with a size of 3.48 MB (a 30.40% size reduction). This shows that reducing the network size does not prevent it from accurately detecting and placing clean geometric shapes.

Next, I wanted to test the model on a more complex image, closer to a real-world use of the pipeline, to see whether it could generalize beyond simple shapes. It was therefore tested on an image of a butterfly. Under these realistic conditions, the model's behavior evolves as one might expect:

- **Drop in repeatability:** The repeatability rate falls to 53.63%. This drop is explained by the fact that the student's filters react strongly to the details of the butterfly's wings, creating secondary keypoints. Visually, the model nevertheless remains effective and detects the insect's main contours well.

- **Precision maintained:** Despite the complexity of the image, the localization error remains well controlled at 0.89 pixel. This score is well below the critical threshold of 3 pixels, which proves that the model remains stable.

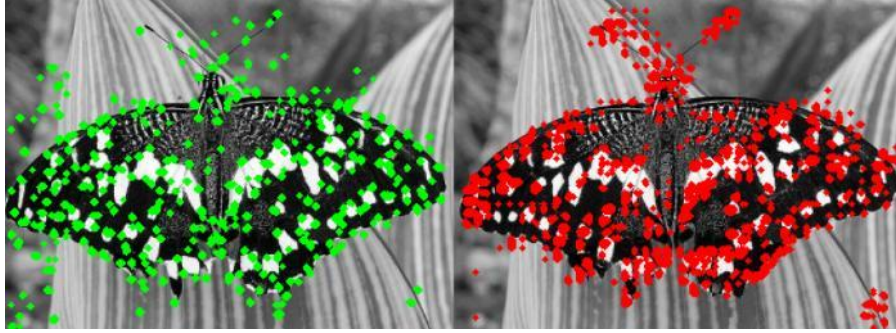


Figure 2 — Points detected on a real image (butterfly): base model (left, green) vs. the 25%-pruned student model (right, red).

Despite the fact that the 25%-pruned model showed higher repeatability, the model that seemed most interesting for the rest of the project was the one pruned at 50%. This choice is justified by its smaller size, but also by its very low MLE, which shows that the points detected by this model are nearly identical to those detected by the base SuperPoint.

Table 1 — Overall comparison of model performance, size, and reduction

Model	Format	Size	Reduction	MLE	Repeatability
Teacher (Original)	Float32	5.20 MB	0.00 %	—	100 %
Student (scale = 0.75)	Float32	3.48 MB	33.08 %	0.89 px	53.63 %
Student (scale = 0.50)	Float32	2.16 MB	58.46 %	0.36 px	45.50 %
Quantized model	INT8	1.26 MB	74.80 %	0.67 px	18.50 %

9) LightGlue Integration and Latent Space Analysis

Once the reduced SuperPoint models were ready, the next step was to continue building the pipeline by integrating the LightGlue matcher. To do this, I wanted to run a first trial with the 25%-pruned model. Indeed, even though the model retained for the final pipeline remains the one pruned at 50%, I preferred to use the model closest to the original SuperPoint for this first integration test, in order to establish a reliable baseline for comparison.

Unfortunately, I once again had to face a failure. While the reference pipeline (original SuperPoint + LightGlue) showed 19 valid matches on our real test images, integrating the distilled model led to a result of 0 valid matches found. Yet I had configured the student model so that its descriptor dimension remained strictly identical to the original ($D = 256$), which ruled out any possibility of a structural or format error from the outset.

I therefore concluded that the problem stemmed from the latent space. Indeed, although the student model shows good geometric repeatability (proving that it visually localizes the same physical corners), the vector space of its descriptors underwent a rotation or mathematical deformation during distillation. However, LightGlue was trained to expect the exact numerical distribution of the original SuperPoint as input. This creates a conflict between the descriptors generated as output by the student model and those expected as input by LightGlue.

Since this student model was the one closest to the base SuperPoint and the integration did not work, I decided not to test the other configurations (notably the 50% version). These would, at minimum, have presented the same technical barriers and the same integration problems.

10) Methodological Pivot and Hardware Constraints

I then had to consider a change of approach for the rest of the project. Indeed, since I could not simply optimize SuperPoint and then optimize LightGlue in isolation before assembling them, I needed to think of a better option.

The first idea that came to mind was to retrain the pipeline end-to-end. The goal was to retrain LightGlue directly on the outputs of my 25%-pruned student model, so that it would learn to adapt to this new latent space. However, working on Google Colab with a single T4 GPU, it seemed unrealistic to retrain a transformer like LightGlue from scratch. I then tried to request access to the university's computing cluster, but unfortunately I was never able to obtain access to that resource.

Faced with this technical and hardware setback, I carried out a comparative analysis of the pipeline's two main models (SuperPoint and LightGlue) to determine where my focus should be in order to successfully complete this project:

Memory Footprint Analysis of the CNN (SuperPoint)

Knowing that the total pipeline weighs 125 MB, SuperPoint's footprint is particularly small. It relies on a classic, compact CNN architecture of only 5 MB. At the scale of a modern embedded system, a fixed weight of 5 MB represents a completely negligible memory load, since it accounts for only 4% of the initial pipeline's total weight.

The Impact of the Transformer (LightGlue)

By comparison, the original, complete 9-layer LightGlue model weighs about 120 MB. It is composed of heavy Transformer attention blocks that generate quadratic complexity, saturating RAM and compute.

This observation forced an immediate logical pivot: since direct integration of our closest student model had failed, and since SuperPoint's size is completely negligible compared to LightGlue's, the optimal strategy at this stage is to temporarily freeze SuperPoint in its original, stable 5 MB version. This freeze guarantees absolute compatibility with the official pipeline and allows us to focus the entirety of our embedded-optimization effort on the LightGlue Transformer, where the potential gains are far greater.

11) Explanation of LightGlue

First of all, before LightGlue, the state of the art was SuperGlue, whose purpose was to analyze two images simultaneously in order to match interest points and reject outliers. SuperGlue is based on a Transformer architecture and uses SuperPoint: the latter provides the locations and descriptors of the interest points, while SuperGlue's role is to compare these points and find the correspondences between the two images.

However, SuperGlue has several drawbacks. First, it is very slow and also very costly to train, requiring significant computing resources (several days, or even weeks, of GPU time). Moreover, its complexity is quadratic in the number of interest points. It has no adaptability whatsoever: regardless of the difficulty of the image pair, it will always use the same amount of computation. Finally, it relies on the Sinkhorn algorithm, which transforms a similarity matrix into an assignment matrix where each point in image A is uniquely (or not) associated with a point in image B. This algorithm, however, is very memory-hungry and does not allow for efficient layer-by-layer training.

It is for all these reasons that LightGlue was proposed. LightGlue is a deep neural network based on a Transformer architecture, which enables joint learning since the model processes both images simultaneously. Its goal is to be a direct improvement over SuperGlue — faster, more efficient, and simpler to train. LightGlue is composed of 9 identical attention layers, each containing a self-attention module, whose purpose is for points within the same

image to communicate with each other to understand the global context, as well as a bidirectional cross-attention module, allowing points in image A and image B to compare themselves and search for their correspondences.

As a Transformer, LightGlue uses Rotary Positional Embedding (RoPE). Unlike absolute positional encodings, it does not use the exact positions of points directly, but rather their relative positions with respect to one another, via rotations. Indeed, when a camera moves, points change absolute position in the image, but their relative geometric relationships remain consistent. After the attention mechanisms, an MLP (an FFN) updates the state of each point (its feature vector) for the next layer.

Where LightGlue really starts to distinguish itself from SuperGlue is that, immediately after the MLP, a Prediction Head is applied. This computes two things: an assignment matrix P (which point goes with which point) by multiplying the descriptors of image A by those of image B, and a confidence score using dual-softmax. Based on this confidence score, LightGlue introduces a stopping criterion (early exit). If the overall confidence score exceeds a predefined threshold (typically between 0.95 and 0.99), the model stops immediately. Thus, for very similar images, confidence scores can be high as early as the first layers, avoiding unnecessary computation.

If this criterion is not met, the model then applies pruning, which uses a sigmoid. That is, if the probability associated with a point falls below a set threshold (often very low, e.g. 0.01, so as not to remove useful points), that point is removed from subsequent computations. The further we go through the layers, the fewer points remain, which greatly reduces the amount of computation needed, since the Transformer's attention has $O(N^2)$ complexity — if, for example, half of the points are removed, the amount of computation is divided by 4.

At the end of the 9 layers, or after an early exit, the model returns, for each point detected in image A, the index of the corresponding point in image B along with the associated confidence score. It also identifies points that have no correspondence.

Regarding training, LightGlue is trained on very large datasets, such as MegaDepth, for which the geometric correspondences between images are precisely known. Training relies on Deep Supervision, meaning a loss is computed after each layer. This makes training more stable and faster, and it is this mechanism that allows pruning and early exit to be so effective. For each layer, a Binary Cross-Entropy (BCE) is used as the loss function, and the total loss is simply the weighted average of the losses computed at each layer. It is this entire process that allows LightGlue to be trained in just a few days, whereas SuperGlue could require several weeks of GPU computation.

12) LightGlue Optimizations

Just-In-Time (JIT) Compilation Optimization

To optimize LightGlue, I first used PyTorch's Just-In-Time (JIT) compilation. As a reminder, JIT compilation transforms a model written in Python code into a frozen, optimized, and ultra-fast format. In particular, it allows the model to be detached from the Python interpreter so it can be run directly in C++ on production servers or mobile applications.

With this method, I obtained a gain of 20.6% in FPS (FPS = the number of images displayed per second), reaching 36.2 FPS with a latency of 27.6 ms (Latency (ms) = the waiting time to process a piece of information), compared to the original LightGlue model, which ran at 30 FPS with a latency of 27.63 ms. This speedup was achieved while maintaining strict 100% geometric fidelity.

Early-Stopping Threshold Optimization

Next, the optimization I found most interesting to test consisted of tuning the early-stopping threshold. Indeed, by applying a more aggressive threshold, the model uses, on average, fewer Transformer layers to validate its correspondences.

The first trial was carried out with a threshold set at 0.90. The results were particularly encouraging and convincing: I obtained a speed of 45.7 FPS and a reduced latency of 21.9 ms, while maintaining a very satisfactory geometric repeatability of 82.4%.

Following these results, I wanted to determine whether it was possible to push the optimization even further by setting the threshold to 0.85. However, this proved a bit too aggressive. Indeed, while it did achieve a speed of 90.1 FPS (i.e., a very low latency of 11.0 ms), representing an extremely significant gain over the base LightGlue, it caused a collapse in geometric fidelity, which fell to 54.9%.

In light of these results, the most interesting model appeared to be the one configured with a threshold of 0.90. In my view, it offers the best compromise, maintaining an acceptable balance between fluidity gains and geometric precision.

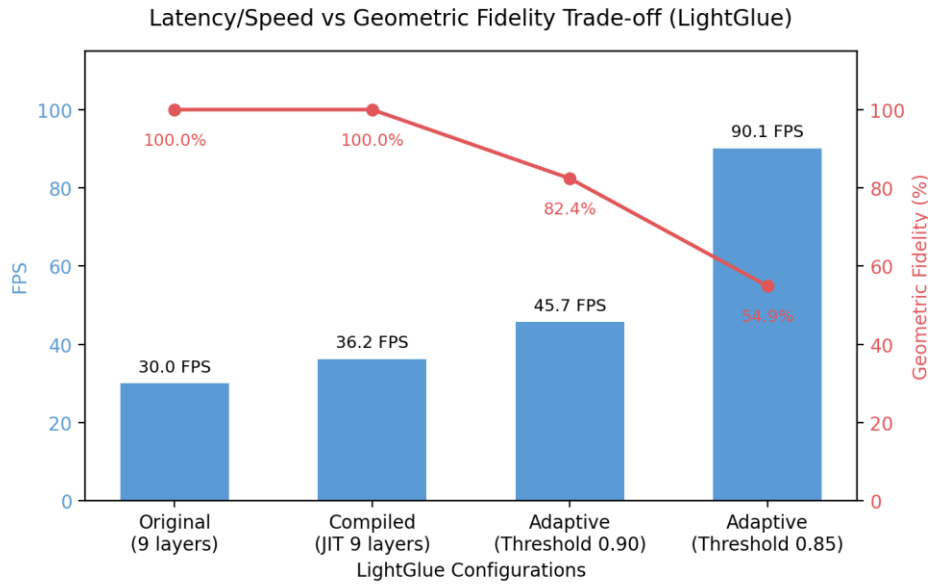


Figure 3 — Latency/speed vs. geometric fidelity trade-off across LightGlue configurations.

13) Conclusion and Future Perspectives

These final tests conclude my experiments on the pipeline combining SuperPoint and LightGlue. Although I would have liked to carry out further investigations, the constraints related to available computing power and time limited the scope of my trials.

Nevertheless, several promising perspectives open up for the continuation of this project, provided a more substantial computing infrastructure is available. The main research direction would rely on knowledge distillation applied directly to LightGlue. Indeed, the authors of the original research paper show that after only 5 Transformer layers, LightGlue already manages to identify 90% of the final correspondences. It would therefore be particularly relevant to design an incremental three-step optimization strategy:

- **Step 1: Structural distillation of LightGlue (5-layer model).** First, the goal would be to create a LightGlue student model limited to 5 Transformer layers. To do this, it would be essential to keep the original SuperPoint model frozen. This approach would make it possible to isolate variables, maintain a reliable comparison baseline, and validate whether this structural reduction is viable.
- **Step 2: Latent space realignment.** Next, the reverse approach would be taken: retraining the complete LightGlue model (9 layers), but feeding it the outputs of the 50%-pruned SuperPoint student model as input. This would allow LightGlue to learn to interpret and correct the deformations of this new latent space.
- **Step 3: Optimal co-distillation of the pipeline.** Finally, if the results of these two experiments proved satisfactory, the culmination of this research would consist of merging the two benefits. A new 5-layer

LightGlue would then be distilled, trained on the outputs of the 50%-pruned SuperPoint, using as the teacher model the complete LightGlue freshly retrained in the previous step.

Such a co-distillation approach would make it possible to obtain an overall pipeline that is much lighter, faster, and perfectly aligned, thereby maximizing performance for deployment on embedded systems.

Conclusion

Although it is difficult to predict with certainty the results of these new avenues of experimentation, the overall size of the pipeline would then go from 125 MB to just over 62 MB. This reduction by half would already represent a genuine success for the integration and deployment of this pipeline within resource-constrained embedded systems.